

# Aqa: 面向纵向扩展 GPU 域中大语言模型的网络加速内存卸载

*Aqa: Network-Accelerated Memory Offloading for LLMs in Scale-Up GPU Domains*

Abhishek Vijaya Kumar  
Cornell University (康奈尔大学)  
abhishek@cs.cornell.edu

Gianni Antichi  
Politecnico di Milano (米兰理工大学)  
gianni@polimi.it

Rachee Singh  
Cornell University (康奈尔大学)  
rachee@cs.cornell.edu

## 摘要

大语言模型（Large Language Model, LLM）的推理受限于 GPU 内存容量。云端托管 LLM 的推理请求数量骤增会耗尽 GPU 内存，导致多个提示词（prompt）为有限资源而相互竞争。现代 LLM 服务引擎通过准入控制（admission control）来应对 GPU 内存有限的挑战，这使得它们在请求突发期间处于无响应状态。我们提出，以时间片对提示词进行抢占式调度是保证 LLM 推理响应及时性的关键，尤其是在高负载且 GPU 内存有限的条件下。然而，抢占提示词的推理会带来高昂的换页开销，从而降低推理吞吐量。我们提出 Aqa——一个 GPU 内存管理框架，它显著降低了推理状态换页的开销，即使在请求模式突发的情况下也能同时实现响应及时与高吞吐的推理。我们在配备 8 块 NVIDIA H100 80G GPU 的服务器上托管多个最先进的、涵盖不同模态的大型生成式机器学习（Machine Learning, ML）模型，对 Aqa 进行评估。与最先进的方案相比，Aqa 将 LLM 推理的响应性提升了 20 $\times$ ，并将单个长提示词上的 LLM 推理吞吐量提升了 4 $\times$ 。<sup>1</sup>

CCS 概念：· 计算方法 → 机器学习；· 网络 → 数据中心网络；数据通路算法。

关键词：大语言模型；生成式人工智能；分页；虚拟内存；GPU 互连；内存卸载

ACM 引用格式：Abhishek Vijaya Kumar, Gianni Antichi, and Rachee Singh. 2025. Aqa: Network-Accelerated Memory Offloading for LLMs in Scale-Up GPU Domains. In Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLoS '25), March 30–April 3, 2025, Rotterdam, Netherlands. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3676641.3715983>

<sup>1</sup> Aqa 的代码发布于 <https://github.com/aquaml>。本作品采用 Creative Commons Attribution International 4.0 许可协议授权。

## 1 引言

近年来，生成式机器学习（ML）模型在从文本摘要到多媒体内容创作等众多任务中的流行度爆发式增长 [15, 38, 49, 53]。这些模型通常托管在虚拟化的多 GPU 云服务器上 [9, 16, 47, 52]，服务器主板上的少量 ML 加速器之间通过高带宽链路互连 [42]。

推理是内存密集型的。生成式 ML 模型的内存占用极其庞大 [5–7, 40]。例如，托管 Llama-405B 模型要消耗 800GB 的 GPU 高带宽内存（High Bandwidth Memory, HBM）。此外，LLM 在利用注意力机制（attention mechanism）处理输入提示词时会生成键值（Key-Value, KV）缓存 [61]。KV 缓存随输入提示词及其生成序列的长度呈平方增长，每个提示词要占用数 GB 的 GPU 内存 [35]。因此，LLM 推理请求数量骤增会耗尽 GPU 内存，多个提示词争抢 GPU 内存时便引发内存争用。

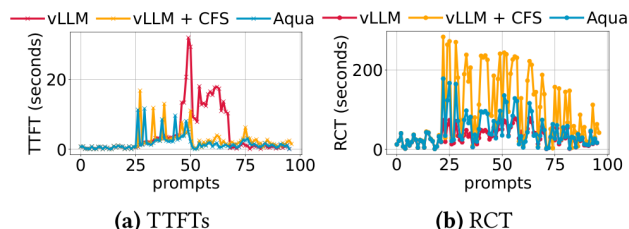


图 1 LLM 推理查询的响应性（以首令牌时间 Time To First Token, TTFT 度量）与吞吐量（以请求完成时间 Request Completion Time, RCT 度量）。由于 vLLM 采用批处理方式处理查询，其 RCT 低（吞吐量高）而 TTFT 高（响应性差）。公平调度查询可改善响应性，但换页开销主导了 RCT。Aqa 通过高速多 GPU 互连（如 NVLink）卸载内存来降低换页开销，在保持低 RCT 的同时实现响应及时的推理。

图中文字说明：(a) 为 TTFT（秒），(b) 为 RCT（秒）；横轴均为提示词数量；图例中 vLLM、vLLM+CFS（加公平调度）、Aqua（即 Aqa）为三种方案。

用排队应对推理请求的突发。现代 LLM 服务引擎通过准入控制应对 GPU 内存有限的挑战 [35, 54]。它们调度一批提示词进行推理，其余的排队等待，直到有空闲 GPU 内存可用。然而，这种带准入控制的批处理范式容易受到队头阻塞（head-of-line blocking）的影响 [21]——处理一批提示词会推迟对队列中等待的其他提示词的响应。

排队导致无响应。我们通过实验证明，把提示词排队会严重损害 LLM 的响应性。我们在一块配备 80GB HBM 的 NVIDIA A100 GPU 上，使用流行的服务引擎 vLLM [35] 托管一个 LLM。我们先向 vLLM 发出 25 个提示词，随后将请求速率加倍并持续一分钟。请求的激增耗尽了 vLLM 的 KV 缓存容量，导致后续提示词进入排队。这使得衡量模型多快开始响应的指标——首令牌时间（TTFT）——骤增 4 $\times$ （图 1a 中的红线）。应对突发的另一种途径是跨

更多 GPU 对模型进行自动扩容 [59]。然而，受虚拟机启动延迟影响，扩容可能需要数分钟，其间模型将处于无响应状态。既然仅仅 3 秒的无响应就会让用户放弃服务 [27]，数十秒的延迟会显著增加流失用户的风险。

**我们的主张：可抢占的推理。**本文主张，以时间片对提示词进行抢占式调度是实现响应及时的 LLM 推理的关键，尤其是在高负载和 GPU 内存有限的情况下。在这种调度方案下，每个提示词都会获得 GPU 计算的时间片，使模型保持响应。抢占式调度器会在时间片边界停止当前提示词的执行，并将其相关状态从 GPU 内存中换出，为下一个提示词腾出执行空间。带时间片的抢占式调度还将为现有推理服务系统实现细粒度公平调度打开大门，这些系统目前仅通过准入控制支持粗粒度公平性 [54]。

**换页开销使抢占变得缓慢。**然而，抢占提示词的推理要求在时间片边界把该提示词的状态从 GPU 内存中换出。如果提示词的推理状态被换页到主机 DRAM [35, 55]，再传回 GPU 内存就会产生很高开销。DRAM 与 GPU 内存之间传输状态的开销下界由外设组件互连标准（Peripheral Component Interconnect Express, PCIe）连接的带宽决定。事实上，我们在 vLLM 中实现了抢占式公平调度器，发现抢占式调度虽然改善了 vLLM 的首令牌时间（图 1a 中的橙线），却使请求完成时间（即完成一个提示词推理所需的时间）增加超过  $2\times$ （图 1b 中的橙线）。

**Aqa：低开开销的可抢占推理。**本文构建了 Aqa——一个 GPU 内存管理框架，它显著降低了推理状态在 GPU 内存换入换出的开销，从而实现高效的抢占式提示词调度。降低后的换页开销让 Aqa 兼得两者之长：既为单个提示词提供响应及时的推理（图 1a），又保持高推理吞吐量（图 1b）。

**Aqa 的关键洞察。**ML 模型的推理可能受限于内存容量、内存带宽或算力，具体取决于模型类型（如视觉还是文本）与推理查询负载 [55, 59, 69]。因此，任何时刻，云集群中总有一些推理作业未充分利用 GPU 内存，而另一些正遭遇内存瓶颈。基于这一洞察，Aqa 把内存分配与计算分配解耦，让 GPU 把空闲内存分享给需要更多内存的 GPU。Aqa 把这类内存卸载限制在云数据中心的纵向扩展域（scale-up domain）内的 GPU 之间。纵向扩展域内的 GPU 通过高带宽 NVLink 相连，使 Aqa 访问空闲内存的速度远快于传统经 PCIe 访问主机 DRAM，从而显著降低抢占与换页开销。在现代数据中心中，多加速器服务器构成一个纵向扩展域。近来，纵向扩展域已扩展到整个机架，例如 NVIDIA Blackwell 的 72 GPU 架构中，高带宽 NVLink 互连全部 72 块 GPU。这些进展把 Aqa 解耦式内存分配的收益延伸到了整个机架 [41]。

**技术贡献。**Aqa 有三个组件：剖析组件 Aqa-profiler 对部署中输入推理负载的特征及所托管 ML 模型的 GPU 内

存利用率进行剖析。基于剖析结果，Aqa-profiler 将模型标记为生产者或消费者，并把信息交给放置组件 Aqa-placer。生产者未充分利用 GPU 内存，可以把富余部分提供给其他模型；消费者受 GPU 内存瓶颈制约，需要更多内存。Aqa-placer 放置模型，使生产者与合适的消费者配对并处于同一 NVLink 域内。最后，Aqa-lib 是一个库，动态管理从消费者卸载到生产者 GPU 内存的张量（tensor）。

**低开开销的抢占式公平调度。**借助 Aqa 在 GPU 之间管理内存，我们为提示词推理构建了抢占式完全公平调度器（Completely Fair Scheduler, CFS）[60]。我们设计了一种新算法，在不同推理阶段的提示词之间划分批次，在保留吞吐量优化的同时实现公平性 [4]。得益于 Aqa 带来的快速空闲内存访问，换页开销极小。公平调度 1) 为所有提示词实现计算资源的公平分配，2) 使模型在高负载和有限 GPU 内存下仍保持响应。我们注意到，公平调度可以吸收短暂的查询突发，或在自动扩容生成新模型实例之前持续应对突发流量。

**结果。**我们在一台配备 8 块最新 NVIDIA H100 80G GPU 的服务器上，托管多个最先进的、涵盖不同模态（如文本、音频、视觉）的大型生成式 ML 模型来评估 Aqa。使用这些 ML 模型与推理负载的代表性分布，我们表明：与最先进的方案相比，Aqa 以首令牌时间衡量，把 LLM 推理的响应性提升了  $20\times$ ；在内存受限的推理 [55, 69] 中，Aqa 把单个长提示词上的推理吞吐量提升了  $4\times$ 。

## 2 背景与动机

ML 推理任务如今对许多组织都至关重要 [49]。终端用户往往把 ML 模型托管在云服务器上，无需部署自己的硬件即可发起查询 [16, 52]。生成式模型可以产出多种媒体内容，包括文本 [8]、图像 [51] 和音频 [34]。以 ChatGPT 为代表的 LLM 尤以文本生成能力著称。

### 2.1 ML 的基础设施

ML 推理的核心是执行若干大型矩阵乘法。因此，ML 推理使用 GPU、TPU 等为快速矩阵乘法优化的专用加速器 [33]。除 ML 加速器支持的浮点运算次数（FLOPs）之外，由于 ML 模型规模快速增长，加速器上的内存已成为支撑快速 ML 训练与推理的关键。生成式 ML 模型拥有数万亿参数，内存占用庞大。为满足模型的内存占用需求，GPU 配备了数十 GB 的高带宽内存（HBM）。通常，GPU 作为外设经 PCIe 连接到主机服务器。主机拥有 CPU 和用于通用计算的 DRAM [43–45]。

**面向 ML 的服务器。**多加速器服务器（如 NVIDIA DGX [46]、Cerebras WSE [22]、Intel Gaudi [32]）已成为更大规模集群和数据中心部署的基本构件，用于 ML 训练与推理。一台多加速器服务器由少量 ML 加速器组成，通

过板载高速电气互连（如 NVIDIA NVLink、Google ICI [33]）相连，成对加速器间可支持高达每秒数百 GB 的带宽。云运营商再用网络织网（fabric）把一排排多加速器服务器连接成数据中心规模的部署 [33, 58]。

**ML 集群中的带宽不对称。**同一服务器内 GPU 之间的互连专为高带宽设计，可达每秒数百 GB [42]；而 GPU 与主机（CPU、DRAM）之间的连接依赖通用 PCIe（50GB/s），在现代部署中显著慢于 NVLink。

## 2.2 LLM 推理期间的内存争用

LLM 把推理查询（即提示词）转换为表示子词单元的令牌（token）。每个令牌被编码为捕捉其语义的多维嵌入向量 [24, 39]。推理期间，这些令牌嵌入被送入 LLM。给定用户提示词，LLM 通过注意力机制 [61] 预测词表中最大的下一个令牌 [20]；该机制将每个令牌的嵌入与学习到的键矩阵、值矩阵相乘，为每个令牌计算键向量和值向量 [24, 39]。一旦生成最大的令牌，它便被追加到序列（包含提示词与先前生成的令牌）中，过程不断重复。

该序列中所有令牌的键向量和值向量——即推理上下文——会被缓存，以支撑高效的未来预测。同时处理的多个序列的键值向量存储在 GPU 内存中的键值（KV）缓存里 [35]。KV 缓存的大小由序列长度与序列数量决定，并随生成令牌数增长。KV 缓存容量受限于 GPU 加载模型权重之后的可用内存。当 KV 缓存超过可用 GPU 内存时，便产生内存争用。

事实上，当模型相对 GPU 过大，或提示词推理上下文（KV 缓存）与模型权重之和超出 GPU 内存时，甚至单个提示词在推理中也可能遭遇饥饿。随着模型规模与所支持的提示词长度迅速增长（如 Llama-405B 与 Gemini 的 1M 提示词），问题将愈发严重。这促使近期工作把上下文从 GPU 换页到 DRAM，仅在需要时才加载 [55, 64, 69]。

**内存争用阻碍缓存优化。**近来，LLM 服务引擎开始缓存多个提示词长公共前缀的推理上下文 [13, 25]。当新到来的提示词消耗内存时，引擎不得不把缓存上下文逐出 GPU，使该优化失效。此时引擎唯一的选择是把推理上下文换页到 DRAM 或远端内存 [37]。

## 2.3 缓解内存争用的现有方式

应对 LLM 推理期间内存争用的一种方式：当推理请求突发到来时，水平扩展托管模型的 GPU 数量 [59]。然而，在更多 GPU 上生成模型实例会带来数分钟的启动延迟 [11, 59]。这些延迟使横向扩展方案只对已被接纳并排队的请求有价值，因为在更多资源就位之前它们将经历无响应。

**换页受制于带宽瓶颈。**换页解决了内存争用带来的问题，但性能不佳，因为 GPU 在等待换页数据时处于空闲。

换页的根本瓶颈在于 GPU 与 DRAM 之间受 PCIe 带宽限制的有限带宽。更高的 PCIe 带宽或许能缓解这一问题，但研究表明 PCIe 带宽的增长率一直很缓慢 [26]。因此，当前把动态上下文卸载到主机 DRAM 的推理服务引擎必须在性能与响应性之间权衡。事实上，为证明这一点，我们在 vLLM 中实现了完全公平调度器，发现公平调度虽然改善了 vLLM 的首令牌时间（图 1a 中的橙线），却使请求完成时间（即完成一个提示词推理所需的时间）增加约 50%（图 1b 中的橙线）。

## 3 Aqa 设计

当推理负载增长时，现代服务系统因 GPU 内存容量争用而显著延长响应时间。图 1 表明，带准入控制的批处理提示词 [35, 54] 会让提示词排队，直到当前批次在 GPU 上执行完毕，导致队头阻塞延迟。即便推理负载只有单个提示词，只要 GPU 内存不足以容纳模型，或 KV 缓存与模型权重之和超过可用 GPU 内存，服务同样可能无响应 [55]。Aqa 通过快速换页降低抢占开销，并实现使用快速换页的抢占式调度器，在 GPU 内存容量争用下保证响应及时的推理。

**Aqa 的快速抢占。**ML 集群中的多加速器服务器被分配给执行各种 ML 模型的推理作业。这些推理作业对 GPU 计算核心、内存带宽和内存容量的利用率各不相同，取决于模型类型（如图像生成还是文本生成）。承载某些推理作业的 GPU 受计算瓶颈制约而留有大量空闲内存，另一些作业则受内存容量瓶颈制约（[57]，第 4 节）。我们利用这一洞察，为推理作业解耦 GPU 上的内存与计算分配。由此，我们可以把有空闲内存的 GPU 上的内存分配给另一块 GPU 上受内存容量瓶颈制约的作业。这使受内存约束的作业能通过高速 GPU 间互连（如 NVLink）访问额外内存，其带宽比 PCIe 高出一个数量级。

Aqa 通过保证响应及时的推理（低首令牌时间）并维持高吞吐量（低请求完成时间），来应对云端托管 LLM 的推理请求突发。Aqa 之能够在突发下实现响应及时且高吞吐的推理，靠的是实现低换页开销的抢占式调度（第 7 节）。Aqa 的快速抢占自然让依赖换页的内存受限推理系统 [55, 64, 69] 以及先进的缓存优化 [13, 25] 受益。第 9 节将表明，除实现响应性外，Aqa 还提升了这些系统的性能。Aqa 有三个主要组件（图 2）：

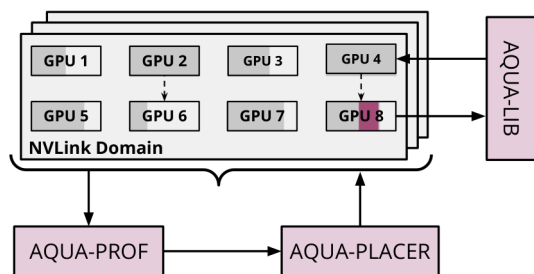


图 2 Aqa 的设计。

图中文字说明：GPU 1~GPU 8 通过 NVLink 域（NVLink Domain）互连；AQUA-PROF（Aqa 剖析组件）、AQUA-PLACER（Aqa 放置组件）、AQUA-LIB（Aqa 库）为三个组件。

- **Aqa-profiler（第 4 节）**。Aqa-profiler 剖析推理负载对任意所托管模型的特征以及相应的 GPU 内存利用率。基于剖析信息，Aqa-profiler 判断 GPU 是有富余内存还是需要更多 GPU 内存来支撑其推理负载。我们把有富余内存的 GPU 称为生产者，需要内存的称为消费者。
- **Aqa-placer（第 5 节）**。我们不依赖附近恰好存在空闲 GPU 内存的机会，而是开发了 Aqa-placer 算法，在集群中放置模型，以最大化经由快速 GPU 间网络卸载内存的机会。Aqa-placer 把模型放置到 GPU 上，使受内存限制的 ML 模型（消费者）托管在内存充裕的 ML 模型（生产者）附近。
- **Aqa-lib（第 6 节）**。推理负载会随时间变化，这意味着推理期间 GPU 内存的需求与供给都是弹性的。这种动态环境使得把推理状态卸载到另一块 GPU 变得困难——那块 GPU 在遭遇推理查询高峰时可能需要收回内存。为此我们开发了 Aqa-lib，一个提供全新 Aqa 张量（Aqa Tensors）抽象的内存管理框架。Aqa 张量是弹性的——允许生产者 GPU 透明地收回曾提供给消费者 GPU 卸载使用的内存。

#### 4 Aqa-profiler：剖析内存消耗

Aqa-profiler 在稳态下结合所服务的负载剖析推理作业当前的 GPU 利用率。利用这些信息，Aqa-profiler 估计 GPU 是生产者还是消费者。我们把 GPU 上留有富余内存容量的 ML 模型称为生产者，受内存容量瓶颈制约的作业称为消费者。

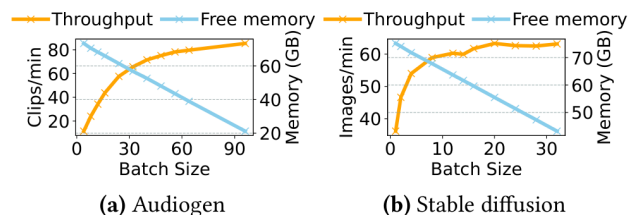


图 3 这些图表明，当音频与图像生成模型达到吞吐量平台期时，A100 80GB GPU 上仍有数十 GB 的空闲内存。事实上，批量大小超过某一点后，吞吐量的增长便趋于平缓。

图中文字说明：(a) Audiogen（音频生成模型），(b) Stable diffusion（图像生成模型）；横轴为批量大小，左纵轴为吞吐量（Clips/min：每分钟音频片段数；Images/min：每分钟图像数），右纵轴为内存（GB）；Throughput（吞吐量）与 Free memory（空闲内存）为两条曲线。

我们的实验表明，不同的生成式 ML 模型争用不同的资源（如计算与内存）。具体而言，视觉与音频生成模型即使在推理吞吐量峰值处也有充裕的可供外借的内存（图 3）。由于需要随时估计空闲内存的大小，Aqa-profiler 通过增大批量大小来测量空闲内存，因为模型的内存消耗随批量大小增长。我们不断增大推理的批量大小，直到模型吞吐量饱和，以此测量模型消耗的最高内存。由于图像和音频的部分响应收益有限，进一步增大批量大小并无必要。若 GPU 在峰值吞吐量下的空闲内存超过阈值（例如 10GB），我们就把该模型视为生产者。

##### 4.1 LLM 能成为内存的生产者吗？

我们“增大推理批量大小直至吞吐量饱和”的策略难以确定 LLM 的真实内存需求。这是因为 LLM 推理能从部分响应中获益，在交互式使用场景中尤其如此。此外，大批量会损害 LLM 的响应时间服务协议（Service Level Agreement, SLA）[4]。因此，我们为 LLM 开发了另一种分类方法。

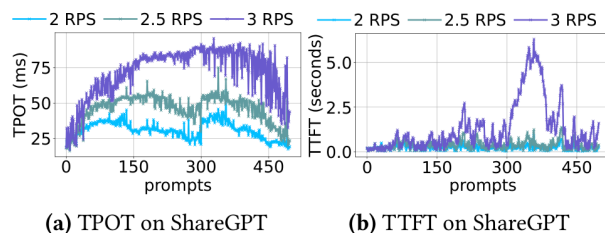


图 4a 展示了 Llama 3.1 8B 模型在 80GB A100 GPU 上的每输出令牌时间（Time Per Output Token, TPOT）。TPOT 随请求速率上升。图 4b 展示首令牌时间（TTFT），随到来请求形成的队列长度增长。

图中文字说明：(a) 为 ShareGPT 数据集上的 TPOT（毫秒），(b) 为 ShareGPT 数据集上的 TTFT（秒）；横轴均为提示词数量；图例 2 RPS、2.5 RPS、3 RPS 为三种请求速率（每秒请求数）。

LLM 的推理有两个主要阶段：预填充（prefill）阶段和解码（decoding）阶段。预填充阶段为提示词中的每个令牌计算键值向量，并对每个令牌与之前所有令牌计算自注意力得分 [61]。预填充阶段的输出是每个令牌一个向量，

再与矩阵相乘生成该层的输出，并保持输入维度传给下一层。解码阶段在每次前向传播中生成新令牌。与处理多个向量的预填充阶段不同，解码阶段每个提示词只处理一个向量。

近期工作提出了把部分输入提示词的预填充阶段与其他提示词的解码阶段合批的推理调度器，以提升整体推理效率[4]。最优批量大小（即块大小，chunk size）取决于所托管 LLM 的 SLA 要求。服务引擎尽可能多地批量处理解码令牌，必要时用新提示词的令牌补充批次。

我们观察到，托管 LLM 的 GPU 常因严格的响应性 SLA 而留有空闲内存，因为批处理受块大小限制。我们用近期工作中交互式用例的 SLA 要求[4]实证了这一点：令牌间隔时间（Time Between Tokens, TBT）P99 为 100 毫秒，首令牌时间（TTFT）低于 1 秒。我们还采用其建议的块大小 512 来处理 ShareGPT [35] 数据集以满足这些 SLA。

**挑战。**给定 SLA、块大小和数据集，Aqa-profiler 搜索能满足 SLA 的最高请求速率，因为它对应最大的内存消耗。穷举整个请求速率空间并不可行也颇具挑战。为此，我们先在一个范围（如 1 - 10 RPS）内二分搜索，再以 0.5 RPS 的步长线性增减搜索局部最大值。据此，Aqa-profiler 输出 LLM 在 SLA 内可支撑的最高流量下的可用内存。在图 4 中，Llama 3.1 8B 模型在不违反 TTFT SLA 的情况下可支撑 2.5 RPS。在 3 RPS 时，队列长度增加，TTFT 达到 5 - 6 秒（图 4b），无法接受。于是，Aqa-profiler 把 2.5 RPS 下的可用内存（40GB）报告为该模型的空闲内存；若该值超过阈值（如 10GB），我们把模型归为生产者。

**不可预测的负载。**与图像生成模型中批次内每个数据点内存消耗相同不同，LLM 推理批次中每个提示词的内存消耗因输入输出令牌长度不同而差异显著。Aqa-profiler 依据各 LLM 部署的输入输出令牌分布采样请求进行剖析[17]。只要请求模式遵循历史分布，其空闲内存估计就保持准确。即便有详尽的剖析，GPU 的内存利用率仍会随不可预测的负载变化而波动。为此，Aqa 能迅速回收内存并归还给生产者（第 6 节）。

**剖析消费者。**若模型不是生产者，Aqa-profiler 会评估其对换页与抢占的需求。若模型已用 Aqa-lib 分配了交换空间，则将其归为消费者，Aqa-profiler 会确定所需的交换空间大小。对内存受限推理与预填充缓存，该空间依据提示词数量、所需的最长序列以及推理上下文所需内存计算。Aqa-profiler 通过不同规模的人为突发（如稳态每秒请求数的  $2\times$ 、 $3\times$ ）经验性地确定 CFS 所需的交换空间（第 9.4 节）。若不需要交换空间，我们视该模型为可提供 0 内存的“生产者”。

**极端情况下的性能。**在所有模型都是内存消费者的极端情况下，Aqa 默认使用 DRAM 进行换页，性能与 vLLM、FlexGen 等基线持平，因为它们同样经 PCIe 换页到 DRAM。

## 4.2 未来数据中心中的 Aqa

ML 基础设施的近期趋势表明，Aqa 的技术非常契合现代数据中心。Aqa 面向多个 ML 作业共存于高带宽互连域（如 NVLink/NVSwitch 织网）的环境。这类域的规模正在扩大，例如 NVIDIA 的 NVL72 机架（72 块 GPU 经 NVSwitch 互连）[41] 与 Google 的 TPU 机架（每个区块 64 个 TPU）[33, 62]。纵向扩展架构的庞大规模提升了多样化 ML 负载共存于同一高带宽域内的可能性。多样化的负载使 Aqa 相比小规模多 GPU 服务器，能把更多的生产者与消费者匹配起来。

## 5 Aqa-placer：最优的模型放置

Aqa-lib 在互连 GPU 之间卸载张量的能力，依赖于这些 GPU 上空闲内存的可用性。Aqa-placer 在服务器集群中把待推理的 ML 模型映射到 GPU，目标是最大化在互连 GPU 上卸载 Aqa 张量的机会。把一个生产者映射到多个消费者是可行的，但 Aqa-placer 刻意不做这种映射，而是将一个生产者对应一个消费者。多个消费者共享一个生产者，可能过度订阅生产者 GPU 的 NVLink 带宽，削弱经 GPU 间网络卸载内存的收益。

**与稳定匹配的差异。**把放置问题建模为简单的稳定匹配会假设任何生产者都能与任何消费者配对。然而在 Aqa 的语境下，只能把生产者与由同一快速服务器级 GPU 间网络（如 NVLink 或 ICI）连接的消费者配对。Aqa-placer 分两步解决模型分配问题：先把模型分配到多 GPU 服务器，再在每台服务器内用简单的稳定匹配为生产者与消费者配对。我们把第一步编码为一个优化问题，产出最优的模型到服务器映射。

**输入与输出。**该优化以集群中的服务器数  $S$ 、每台服务器的 GPU 数  $G$ 、模型数  $N$ ，以及每块 GPU 的高带宽内存总容量  $G_{mem}$  为输入。受当前多 GPU 服务器启发[43, 46]，我们做简化假设：同一服务器内所有 GPU 的硬件规格相同（如计算核心与内存）。关键输入是每个模型的内存需求  $R_m$ ，由 Aqa-profiler 确定。模型的内存需求为正表示生产者，为负表示消费者。例如，图 3a 中 Audiogen 的  $R_m = 20\text{GB}$ 。编码的输出是指示变量  $x_{m,s}$ ，当模型  $m$  映射到服务器  $s$  时取 1。为支持分布式服务，我们为每个模型分片  $i$  赋予指示变量  $x_{i,s}$ 。 $P$  是映射列表，每项存储一个模型的所有张量并行分片集合。

模型到 GPU 的分配须满足以下约束：

每个模型只属于一台服务器。每个模型必须映射到唯一服务器：

$$\sum_{s=1}^S x_{m,s} = 1, \forall m \in M \quad (1)$$

张量并行分片在同一服务器。张量并行（Tensor Parallel, TP）分片通常放置在同一服务器，流水线并行分片放置在不同服务器 [4, 63]。据此，我们把一个模型的所有 TP 分片映射到单台服务器：

$$x_{p[j],s} = x_{p[j+1],s} \quad \forall j \in \{0 \dots \text{len}(p)-1\}, \forall p \in P \quad (2)$$

模型数受每台服务器 GPU 数限制。在 Aqa 中，一个 ML 模型分片恰好放置在一块 GPU 上。若一个分片映射到一块 GPU，则映射到一台服务器的分片数不应超过该服务器的 GPU 数：

$$\sum_{m=1}^M x_{m,s} \leq G \quad \forall s \in \{1 \dots S\} \quad (3)$$

内存供需平衡。我们定义  $mem_s$  为映射到服务器  $s$  的各模型内存需求 ( $R_m$ ) 之和。注意  $R_m$  可正（消费者）可负（生产者），因此  $mem_s$  反映模型映射后服务器上的净可用内存：正表示富余，负表示缺口。

$$mem_s = \sum_{m=1}^M x_{m,s} \cdot R_m \quad \forall s \in \{1 \dots S\} \quad (4)$$

消费者与生产者数量平衡。尽管公式（4）保持了服务器上的内存使用平衡，编码仍可能找到把一个生产者与多个消费者映射到同一服务器的解。我们统计映射到服务器的模型净数量，消费者计 -1、生产者计 +1（公式 5）。我们寻找  $eq_s$  接近 0 的解，以避免生产者带宽被过度订阅：

$$eq_s = \sum_{m=1}^M x_{m,s} \cdot t \quad \forall s \in \{1 \dots S\}, t = |R_m| / R_m \quad (5)$$

目标。Aqa-placer 的目标是：把模型映射到服务器后，各服务器之间不浪费 GPU 内存（公式 4），且每台服务器的消费者与生产者数量均衡（公式 5）。我们取服务器上内存利用 ( $mem_s$ ) 与模型数 ( $eq_s$ ) 两者的最大值并最小化其最大和。为使目标两部分单位一致，我们把  $eq_s$  乘以服务器的 GPU 内存：

$$\max_s(mem_s) + G_s \cdot \max_s(eq_s) \quad (6)$$

我们用商用优化求解器 Gurobi [31] 编码该算法，发现对于多达 128 块 GPU 的集群，它在 5 秒内即可达到与最优解相差 2% 以内的解。

**多久运行一次 Aqa-placer？** 当生产者负载变化时，Aqa-lib 会迅速为生产者回收内存，限制消费者的快速换页。这防止了生产者性能下降，但集群状态本身可能不再是最大化卸载机会的最优状态。基础设施提供商因此须在维持次优卸载与重跑 Aqa-placer（通过迁移重映射作业以获得更好的共享机会）之间权衡。本文未探讨这一权衡，留作未来工作。

## 6 Aqa-lib：弹性内存管理

Aqa-lib 为 GPU 内存中可迁移的张量实现了一种新颖抽象，称为 Aqa 张量（Aqa Tensors）。Aqa 张量可以借助快速 GPU 间网络卸载到其他 GPU 的内存。内部地，Aqa-lib 依据服务器内所有其他 GPU 的内存利用率与推理负载，决定何时迁移 Aqa 张量以及迁移到哪块 GPU 的内存。ML 模型开发者无需感知 Aqa 张量物理位置的变化。我们把 Aqa 集成到两个流行的 ML 服务引擎 vLLM [35] 与 FlexGen [55] 中，集成并不需要大改其代码库（第 8 节）。

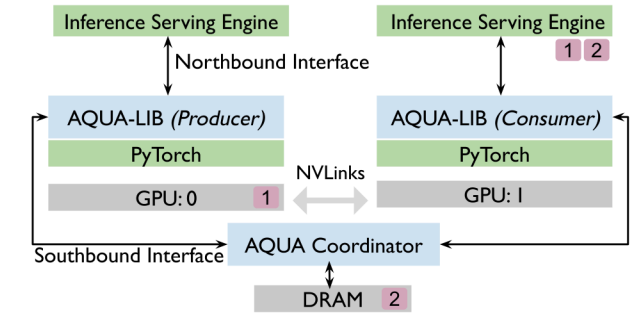


图 5 Aqa 的设计。GPU 1 托管着一个消费者。Aqa-lib 知晓 GPU 0 是生产者。Aqa-lib 允许 GPU 1 上的模型分配 Aqa 张量并卸载到 GPU 0 的 HBM（如编号 1 的粉色框所示）。若 GPU 0 只有够卸载一个张量的内存，Aqa-lib 便退回到主机 DRAM（如编号 2 的粉色框所示）。

图中文字说明：编号 1、2 对应正文中“卸载到生产者 GPU 内存”与“退回主机 DRAM”两条路径。

**Aqa-lib 的接口。** 多 GPU 服务器的每块 GPU 上都运行一个 Aqa-lib 实例（图 5）。Aqa-lib 以 Aqa-profiler 生成的剖析结果初始化，据此判断该 GPU 是否有足够空闲内存充当生产者。Aqa-lib 实现了北向与南向接口。服务引擎通过北向接口与 Aqa-lib 交互，并以可调整频率向 Aqa-lib 共享推理负载（如每秒请求数）与推理上下文规模等指标。若该 GPU 是生产者且服务引擎报告推理负载上升，Aqa-lib 将回收已卸载的内存；当负载回落到 Aqa-profiler 剖析时确定的稳态值时，Aqa-lib 再把内存提供给消费者。当 Aqa 从生产者处提供内存或收回内存时，它通过北向接口告知推理服务引擎该事件后其拥有的内存量。南向接口使 Aqa-lib 能与由 Aqa 协调器维护的集中式线程安全数据存储通信。协调器跟踪来自消费者 GPU 的内存卸载请求与来自生产者 GPU 的内存供给。

**中央协调器。** 中央协调器记录 HBM 的消费者与生产者。消费者 GPU 需要把张量卸载到潜在生产者 GPU 的 HBM 内存。协调器程序提供一组 REST 端点：Aqa-lib 用这些端点为消费者 GPU 登记内存请求、为生产者 GPU 登记空闲内存供给；协调器还提供 REST 端点允许生产者

GPU 在需要时收回内存。一块 GPU 起初可能是生产者，但当其推理负载特征变化时，也可能收回全部已提供的内存。

**分配 Aqa 张量。**ML 模型导入 Aqa-lib 并用 Aqa-profiler 生成的剖析结果初始化。ML 模型用 Aqa-lib 实例化 Aqa 张量。Aqa-lib 通过南向接口，经由相应的 REST 端点向协调器登记这次分配请求（第 8 节）。协调器记录可能已登记为生产者的 GPU。为消费者 GPU 选定哪块 GPU 作为生产者，是由 Aqa-placer 在模型于消费者 GPU 上开始执行之前显式决定的（第 5 节）。协调器把卸载张量位置的引用返回给消费者的 Aqa-lib。我们注意到，若系统中不存在生产者 GPU，Aqa-lib 会像以往工作 [35, 55] 一样退回用 DRAM 卸载张量。释放已分配张量的逻辑与此类似：消费者的 Aqa-lib 实例通知协调器，由后者集中管理 Aqa 张量的记账。

**回收 Aqa 张量。**在生产者 GPU 上，Aqa-lib 轮询北向接口获取该 GPU 当前的内存利用率与负载统计。据此，当生产者 GPU 上的推理负载上升时，Aqa-lib 允许生产者收回被卸载的内存。这发生在 Aqa-lib 发现推理服务引擎报告的负载增加之时。Aqa-lib 用 REST API 告知协调器生产者 GPU 想收回内存。

**Aqa-lib 控制循环。**在消费者 GPU 上，Aqa-lib 的控制逻辑每隔若干次推理迭代检测一次是否有已卸载的 Aqa 张量需要释放。该检测需要 Aqa-lib 调用协调器的 REST API。这一检测确保：若生产者已登记收回内存的请求，消费者能及时响应。需要说明，集中式协调器的数据存储是线程安全的，以防多块 GPU 同时修改引发意外行为。在我们的设计中，Aqa-lib 的开销很低，因为与中央协调器的通信并不频繁——仅在可配置的若干次推理迭代之后进行一次。Aqa-lib 的更多实现细节见第 8 节。

## 7 用 Aqa 对提示词进行公平调度

如今推理服务系统中调度器的默认行为是：仅当 GPU 有足够内存容纳推理上下文时才接纳新请求 [4, 35]。VTC [54] 等调度器通过准入控制提供粗粒度公平性，防止高查询速率的用户占满全部计算周期。然而，当合法用户数量激增并超出所分配硬件的服务能力时，这种粗粒度准入控制仍会导致饥饿。若 GPU 内存已被正在推理的提示词占满，任何后续请求都只能等待或饥饿，直到 GPU 内存腾出足够空间。即便有自动扩容，提示词仍可能饥饿，因为新虚拟机的就位要以分钟计 [11]。

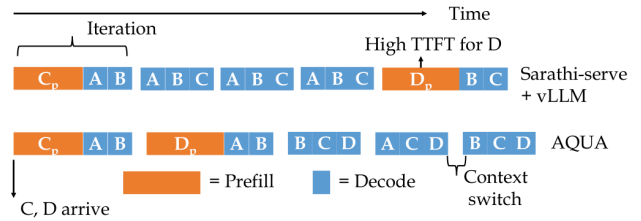


图 6 内存争用下 Aqa 实现的 CFS：5 个提示词中一次只有 3 个能装入 GPU。

图中文字说明：Sarathi-serve + vLLM 与 AQUA（即 Aqa）两种方案的时间线；方格内字母 A、B、C、D 为提示词， $C_p$  表示提示词 C 的预填充（Prefill）， $D_p$  为 D 的预填充，其余为解码（Decode）；C、D arrive 表示提示词 C、D 到达；Context switch 表示上下文切换；High TTFT for D 表示 D 的 TTFT 很高；Iteration 为迭代序号，Time 为时间。

**批处理与分时：推理调度的两种方式。**回顾一下，最先进的调度算法把预填充计算块与其他提示词的解码计算合批以提高计算效率（第 4 节）。该算法在合批时优先解码，当多个提示词同时到达时，便导致其中一部分提示词饥饿。图 6 中，C、D 同时到达，但 D 一直饥饿到 A 执行完毕，因为 GPU 内存一次只能容纳 3 个提示词。这与操作系统行为形成对比：操作系统在负载下用完全公平调度器（CFS）[60] 与上下文切换管理受限资源。上下文切换分两步：抢占，把当前进程的状态（寄存器、页表等）保存到 DRAM；加载，把下一个进程的状态装入 CPU。

**挑战。**简单照搬操作系统调度中的时间片策略——优先调度获得计算时间最少的进程——会破坏近期的分块预填充优化 [4]。这些优化利用了“预填充受计算限制而解码受内存带宽限制”的事实，允许预填充与解码令牌合批，等效提供“免费”的解码。按已生成令牌数最少来分时 GPU 只会批量处理预填充提示词（它们生成的令牌数为零），抵消这一收益。

**Aqa 的批次划分算法。**在 Aqa 中，我们在保留分块预填充收益的同时实现公平调度。给定批量大小  $b$ ，我们把它划分为  $p$  个预填充令牌与  $d$  个解码令牌。划分的关键洞察是把  $d$  设为其上界，即 GPU 内存能容纳的最大提示词数。Aqa 先用已计算预填充令牌最少的预填充提示词填充  $p$ ，用已生成令牌最少的解码提示词填充  $d$ ，再把  $d$  的剩余槽位分配给  $p$  中的提示词。这确保在没有预填充时分充分利用 GPU 容量，而在有预填充时利用分块预填充优化。当 GPU 内存耗尽时，Aqa 停止填充批次。

为简单起见，我们把时间片度量为推理迭代次数，因为给定批量大小的每次迭代耗时相同 [66]。Aqa 每  $k$  次迭代或每当一个请求完成时重排批次，把不属于下一批的提示词换出、把不在 GPU 上的提示词换入。图 6 展示了 Aqa 如何在预填充与解码提示词之间公平分配计算，降低提示词 D 的 TTFT。

**高效的上下文切换。**直接把 Aqa 张量当交换空间用并非最优，原因有二。其一，vLLM 把与某一层相关的所有提示词的键值张量存成一个张量，因此给定令牌的键值张量分散在不连续的不同层张量中，导致向已卸载的 Aqa 张量进行多次小规模拷贝。其二，NVLink 带宽对小数据传输不利：例如在两块 A100 GPU 之间，4MB 缓冲区的拷贝带宽是 50GB/s，而 64MB 缓冲区可达 200GB/s。我们通过把较小的张量先汇聚（gather）到 GPU 上的一个临时张量，再将其拷贝到卸载的 Aqa 张量来解决这一挑战。类似地，Aqa-lib 把卸载的数据拷贝到临时张量，再散射（scatter）到各自较小的张量。

最后，服务引擎可以查询 Aqa-lib 获取张量位置，当张量位于 DRAM 时从 CFS 退回 FCFS，以避免高昂的上下文切换开销。

**其他调度策略。**注意，任何需要交换的策略都可以借助 Aqa 实现，比简单换页到 DRAM 更快。例如，优先级公平可以用 Aqa 实现：把低优先级提示词抢占到由 Aqa 张量承载的交换空间。只要有可能，Aqa-lib 就把这些张量放置到互连 GPU 的内存上。

## 8 实现

我们把 Aqa-lib 实现为可 pip 安装的 Python 库，基于 PyTorch [1] 张量，共 500 行代码（LOC）。该库提供把标准 CPU 张量转换为 Aqa 张量的函数，以及为计算访问取回相应 GPU 或 DRAM 指针的函数。内部地，Aqa-lib 与协调器通信。我们把该库的北向接口集成到服务引擎中，允许用户指定 GPU 内存的提供与保留，由 Aqa-profiler 管理。

在 vLLM v0.5.3 中，我们用 150 行代码实现了 CFS（第 7 节），另用 200 行代码实现了一个 CUDA 内核，高效地把小缓冲区汇聚成较大缓冲区以进行 NVLink 传输。我们引入了带唯一 ID 的新预填充缓存 API，CFS 与预填充缓存共用同一交换空间进行上下文抢占。Aqa-lib 被集成进 vLLM 与 FlexGen [55]，用不到 50 行代码把交换空间从 PyTorch CPU 张量改为 Aqa 张量管理。Aqa 张量的一大挑战是在不引发数据损坏的情况下把张量从 GPU 移到 DRAM。我们凭借如下洞察解决：推理引擎的执行循环边界处没有 GPU 计算，可安全传输。

我们用 Gurobi [31] 以 700 行代码实现 Aqa-placer。Aqa 协调器是一个套接字服务器，提供 REST API 供生产者与消费者分别提供/收回与分配/释放内存，共 250 行代码。

## 9 评估

本节将展示，Aqa 的快速抢占显著提升了响应性与吞吐量。

**实验测试床。**我们的实验使用两套测试床。主测试床为一台配备 8 块 NVIDIA H100 GPU（每块 80GB 内存）的服务器，GPU 之间经 NVLink 与 NVSwitch 互连（450GB/s 带宽），并经 PCIe Gen5（50GB/s）连接 DRAM，服务器内存为 1.5TB。辅助测试床为一台配备 2 块 NVIDIA A100 GPU（每块 80GB 内存）的服务器，GPU 直接经 NVLink 相连（300GB/s），并经 PCIe Gen4（25GB/s）连接 DRAM。我们用 H100 测试床做端到端测试，用 A100 服务器做微基准以控制成本。

### 9.1 模型与负载

**数据集。**我们用 parti-prompts [28] 数据集评估视觉生成模型，用 Audiogen 的默认提示词评估音频生成模型。沿用近期研究 [4]，我们用 sharegpt [35] 与 arxiv 摘要 [14] 数据集评估 LLM。

**分布式服务。**我们采用 vLLM 的默认策略：只要模型装得进一台服务器，推理期间就用张量并行把模型分布到多块 GPU [35, 56]，并通过改变模型切分的 GPU 数来代表多样化负载。具体而言，Llama 70B 可以用张量并行部署在 2 块 GPU 上，也可以在流量持续偏高时以更高成本扩展到更多 GPU。例如，在 H100 GPU 上运行 ShareGPT 数据集时，从 2 块增加到 4 块 GPU 能让 Llama 70B 承载十倍的请求，因为 4 块 GPU 在加载模型权重后有充裕内存。但如果机构预期流量较低、偶有突发，用更少 GPU 并随突发弹性应对更具成本效益。我们的评估同时研究这两种配置，并允许 Aqa-profiler 动态判定其应作为生产者还是消费者。

**生产者负载。**对音频与视觉生成模型，我们以最先达到峰值吞吐量的批量大小执行模型。对 Llama 8B 与 70B 模型，我们用 ShareGPT 数据集、按交互式负载 SLA [4] 运行，并选取使队列长度保持稳定的请求速率，即请求到达速率等于请求完成速率。实验中采用与近期分块预填充工作 [4] 相同的块大小。

**消费者负载。**我们评估三类消费者负载：第 7 节引入的需要公平调度（CFS）的突发负载、预填充缓存负载，以及内存受限的长提示词负载。

**突发期间的公平调度。**我们用两个模型评估突发负载：单 GPU 上的 Yi-34B 与经张量并行分布到 2 块 GPU 的 Llama-70B。我们先确定两个模型在 ShareGPT 数据集上的稳定请求速率，再把请求速率加倍持续一分钟以制造突发。假设为新实例供给资源以应对突发的时间上限为 1 分钟，我们实现了近期工作 [59] 提出的最先进方法，在自动扩容的同时做负载均衡。我们每 5 秒跟踪一次请求到达速率，一旦速率上升就触发自动扩容通知。负载生成器监听该通知并模拟负载均衡器的行为：先等待一分钟让虚拟机启动，再把新到来的请求重定向到一个哑 HTTP 服务器，直

到突发中的请求数量都被重定向完毕。此后所有实例恢复以稳定速率服务请求。我们用哑服务器而非虚拟机，以避免额外占用 GPU 并节省成本。

**预填充缓存。**我们使用以 vLLM 服务于 2 块 GPU 的 Yi-34B-200K 模型，使其有足够 GPU 内存容纳大提示词，并以此评估预填充缓存。我们选取一个 10 万令牌的大提示词进行缓存，并将其与 ShareGPT 数据集交织发出。每次发出该提示词时，只修改最后 256 个令牌，以复用被缓存的提示词，模拟长上下文问答等用例。

**长提示词。**我们使用以 FlexGen [55] 服务的 OPT-30B 模型，评估加载模型权重后超出 GPU 可用内存的长提示词。FlexGen 通过“抢占上一层的上下文、加载下一层的上下文”把提示词的推理上下文卸载到 DRAM。我们测得 arxiv 摘要数据集的提示词长度中位数超过 7000 令牌，因此采样长度为 8192 的提示词评估 FlexGen——其评估脚本要求实验全程使用单一提示词长度参数。

## 9.2 基线

我们用两个基线评估 CFS 负载：vLLM 默认调度器 (FCFS) 与带 CFS 的 vLLM。尽管 VTC [54] 等非抢占式准入控制调度器能防止恶意用户垄断计算资源，但在合法用户激增时仍会遭遇排队延迟。为模拟该行为，我们的 vLLM 基线把突发中的每个请求都当作新的独立用户，刻画其无响应性。因此，图中 vLLM 曲线同样代表 VTC 的性能。我们以“我们实现的、把上下文抢占到 DRAM 的带预填充缓存 vLLM”作为预填充缓存的基线，以“把上下文抢占到 DRAM 的原版 FlexGen”作为长提示词负载的基线。除专门评估 CFS 外，所有实验均使用默认调度器。

| 模型                   | 负载               | 引擎         | GPU 数 | 类型 |
|----------------------|------------------|------------|-------|----|
| Llama3.1 70B         | ShareGPT         | vLLM + CFS | 2     | C  |
| Yi-34B-200K          | ShareGPT + 预填充缓存 | vLLM       | 2     | C  |
| OPT-30B              | 长提示词             | FlexGen    | 1     | C  |
| Llama3.1 8B          | ShareGPT         | vLLM       | 1     | P  |
| Llama3.1 70B         | ShareGPT         | vLLM       | 4     | P  |
| SD-3、SD-XL、Kandinsky | Parti prompts    | Diffusers  | 1     | P  |
| MusicGen、AudioGen    | 音频描述             | PyTorch    | 1     | P  |

表 1 模型、负载、服务引擎及其类型 (P = 生产者, C = 消费者)。SD = Stable-Diffusion。

**评估指标。**我们使用首令牌时间 (TTFT)、每输出令牌时间 (TPOT) 以及以令牌/秒计的解码吞吐量或生成吞吐量。

## 9.3 端到端评估

我们在 8 GPU 的 H100 服务器上运行端到端评估，并把集群规模设为 16 以控制成本。

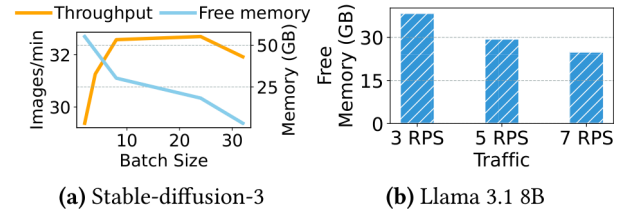


图 7 图 7a 展示 Aqa 为 Stable-diffusion-3 生成的剖析：该模型在批量大小约 8 张图像时达到峰值吞吐量，GPU 上有近 30GB 未用内存。图 7b 表明：在符合 SLA 的 5 RPS 稳态速率下，Llama 8B 有超过 25GB 空闲内存。

图中文字说明：(a) Stable-diffusion-3，横轴为批量大小，左纵轴为吞吐量 (Images/min)，右纵轴为内存 (GB)；(b) Llama 3.1 8B，横轴为流量 (Traffic)，图例 3 RPS、5 RPS、7 RPS 为请求速率，纵轴为空闲内存 (GB)；Throughput (吞吐量) 与 Free memory (空闲内存) 为曲线标注。

**模型如何被分类？**图 7a 给出 Aqa-profiler 为 H100 GPU 上 Stable-diffusion-3 模型生成的轨迹：剖析器在推理期间不断增大批量大小直到吞吐量饱和，并记录相应的空闲 GPU 内存。图 7b 给出 Llama 3.1 8B 模型的轨迹。沿用先前工作 [4, 35]，我们选取 TTFT 低于 1 秒、令牌间延迟 (inter-token latency, ITL) 低于 100 毫秒的 SLA，面向批量大小为 512 的交互式负载。基于该 SLA 与 ShareGPT 输入请求分布，剖析器测试多种请求速率并记录服务期间的可用空闲 GPU 内存。实验发现，该 LLM 在 SLA 内可支撑的最高流量为每秒 5 个请求 (5 RPS)，见图 7。我们对实验设置中的其他模型做同样剖析，以分类生产者与消费者。

**把模型放置到 GPU。**完成剖析后，我们从表 1 的模型中按 10% 采样音频模型、15% 采样视觉模型、其余采样 LLM（以体现 LLM 更高的流行度），直到耗尽可用 GPU。我们以采样出的模型及其 Aqa-profiler 内存剖析结果执行 Aqa-placer。

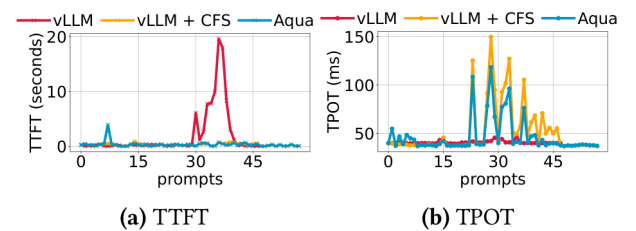


图 8 图 8a 展示以张量并行运行于 2 块 GPU 的 Llama 3.1 70B 按序到来请求的 TTFT，请求采样自 ShareGPT。图 8b 展示按序请求的 TPOT。Aqa 最小化了 TTFT，同时与 vLLM + CFS 相比 TPOT 至多低 1.5×。

图中文字说明：(a) 为 TTFT (秒)，(b) 为 TPOT (毫秒)；横轴均为按序到达的提示词数量；图例 vLLM、vLLM + CFS、Aqa (即 Aqua) 为三种方案。

**CFS 从 Aqa 获益多少？** Aqa-placer 把运行于 2 块 GPU 的分布式 CFS 负载 (Llama 3.1 70B) 与两个单 GPU 模型 (Llama 8B 与 Audiogen) 匹配。CFS 负载的两个分片用 Aqa-lib 分配交换空间，透明地放置在运行 Llama 8B 与 Audiogen 的 GPU 上。我们以稳定请求速率启动 CFS 负载，发出 25 个提示词后，负载生成器把请求速率加倍一分钟。基线 FCFS 调度器应对这一激增时，TTFT 高达 20 秒 (图 8a)。vLLM 上的 CFS 策略降低了 TTFT，但因向 DRAM 的抢占开销而出现高 TPOT (图 8b)。相比之下，Aqa 不仅把 TTFT 降低了 20 $\times$ ，还把 TPOT 相比 vLLM + CFS 至多降低 1.5 $\times$ 。Aqa 同时缩短了突发影响的持续时间：图 8a 中，vLLM + CFS 的 TPOT 在第 25 - 48 个请求之间保持高位，而 Aqa 仅在 25 - 40 之间。这一提升源于 Aqa-lib 的快速抢占——经 NVLink 透明地把数据卸载到相邻 GPU。图 8 还表明，在没有内存争用时（前 25 个请求与第 50 个请求之后），CFS 的行为与 FCFS 完全一致。

**预填充缓存上的改进。** Aqa-placer 把运行于 2 块 GPU 的分布式 Yi-34B-200K 与两个单 GPU 生产者 Stable-Diffusion-3 (SD-3) 和 Kandinsky (Kand) 匹配。我们以稳定请求速率启动，随后发出一个 10 万令牌的提示词，并用 API 为其除最后 256 个令牌外的全部内容缓存推理上下文。之后，10% 的请求以缓存令牌为前缀、接不同的 256 令牌后缀，其余请求采样自 ShareGPT。服务引擎最初计算该上下文约需 10 秒，将其缓存到交换空间，并在携带相同前缀的请求到来时装入 GPU。图 9a 追踪缓存后 30 个稳态请求的情况。

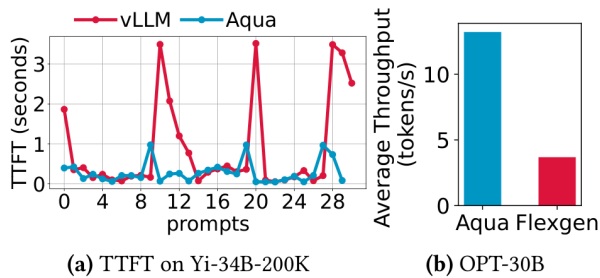


图 9 图 9a 展示按序发给以张量并行服务于 2 块 GPU 的 Yi-34B-200K 的请求的 TTFT，尖峰源于从交换空间加载缓存上下文。图 9b 展示 OPT-30B 在 1 块 GPU 上服务的解码吞吐量。Aqa 显著提升了两种负载的性能。

图中文字说明：(a) 为 Yi-34B-200K 上的 TTFT (秒)，图例 vLLM 与 Aqua (即 Aqa)；(b) 为 OPT-30B 的平均吞吐量 (tokens/s，令牌/秒)，图例 Aqua Flexgen (即 Aqa 与 FlexGen 基线)。

基线在每次缓存命中时都会出现高达 4 秒的 TTFT 尖峰，且后续请求因与长提示词合批、等待其上下文装入 GPU 而同样面临高 TTFT。Aqa-lib 最小化了从交换空间到 GPU 的上下文加载时间，把长提示词的 TTFT 降低了 3.5 $\times$ ，也降低了与缓存提示词合批的后续请求的 TTFT。

**内存受限推理上的改进。** Aqa-placer 把单 GPU 上的 OPT-30B 与 Llama 3.1 8B 匹配。我们以 FlexGen 运行 OPT-30B——该系统用于低资源的离线推理场景 [48]，旨在通过高效地把上下文换页到 DRAM 来提升系统吞吐量。我们在长度 8192 的一串提示词上测量 FlexGen 的生成吞吐量并持续 10 分钟，报告跨提示词的平均吞吐量。8K 令牌提示词生成的上下文在加载模型权重后无法装入 GPU，系统开始换页以执行推理。图 9b 表明，在整个实验期间，Aqa 相比换页到 DRAM 的基线持续取得 4 $\times$  的生成吞吐量提升。

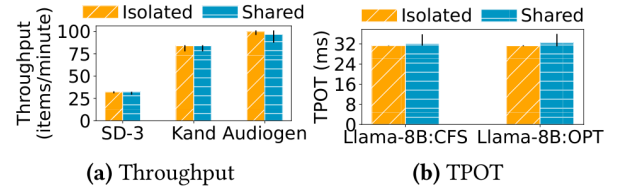


图 10 图 10a 表明，音频与图像生产者与消费者负载共享内存时，吞吐量影响可以忽略。Isolated 柱形为无消费者时的生产者性能。图 10b 表明，内存共享使 2 个 Llama 8B 实例的平均 TPOT 增加约 4.5 毫秒。

图中文字说明：(a) 为吞吐量 (items/minute)，横轴 SD-3、Kand、Audiogen 为三个生产者模型，图例 Isolated (隔离)/Shared (共享)；(b) 为 TPOT (毫秒)，横轴 Llama-8B:CFS、Llama-8B:OPT 为两个 Llama 8B 实例。

**共享内存对生产者有何影响？** 我们把先前实验中与消费者共享过内存的全部生产者放在无消费者的环境中执行，并与共享内存时的性能对比。图 10 表明，共享内存对各种模态生产者的影响都很小：audiogen 的吞吐量差异小于 5%，视觉生成模型则没有差异。我们用平均 TPOT 度量 LLM 生产者的性能，因为它刻画执行一次推理迭代的时间。图 10b 表明共享内存对 TPOT 的影响约为 4.5 毫秒。如近期工作 [2] 所示，消费者访问生产者的 GPU 内存可能引发内存带宽争用。我们认为，LLM 与 audiogen 性能下降的原因在于它们依赖 Transformer 层，而后者已知受 GPU 内存带宽制约 [4]，任何带宽争用都会降低性能。

contention that reduces the performance.

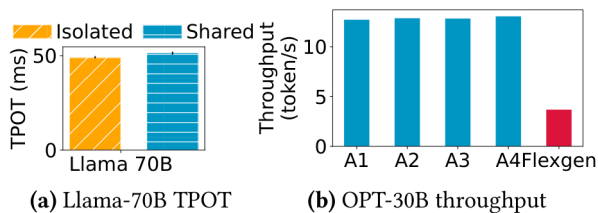


图 11 图 11a 表明，在 10 分钟内，内存共享对以张量并行服务于 4 块 GPU、运行 ShareGPT 数据集的 Llama 70B 的平均 TPOT 影响为 5%。图 11b 展示同一时段 4 个 OPT-30B 实例的吞吐量，持续比基线高 4×。（A = Aqa）

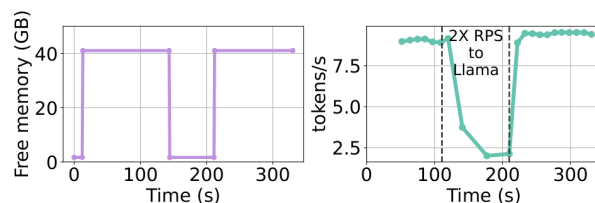
图中文字说明：(a) 为 Llama-70B 的 TPOT（毫秒），图例 Isolated（隔离）/Shared（共享）；(b) 为 OPT-30B 吞吐量（token/s），横轴 A1-A4 为 4 个 Aqa 实例，Flexgen 为基线。

**对分布式生产者有何影响？** 由于 Aqa-placer 未给分布式 LLM 生产者匹配任何消费者，本实验专门研究经 NVLink 共享内存对分布式 LLM 性能的影响。张量并行推理需要经 NVLink 进行 GPU 间通信，消费者访问生产者 GPU 上的内存也要走同一路径，可能影响性能。为此，我们把 Llama 3.1 70B 以张量并行运行在 4 块 GPU 上，并把 4 个 OPT-30B 实例运行在另外 4 块 GPU 上。每块服务 Llama 的 GPU 有 15GB 空闲空间，各作为一个 OPT-30B 的生产者——后者推理 8K 令牌提示词需要 10GB 交换空间。我们选 OPT-30B 做本实验，因为它频繁利用 NVLink 换页推理上下文。

实验运行 10 分钟，Llama 处理稳定请求，OPT-30B 从长提示词生成令牌。图 11 表明，共享内存给 Llama 带来的 TPOT 开销可以忽略，为 5%。我们调查了低开销的原因并确定两点：其一，张量并行期间的通信量只有几 MB；其二，这种通信发生在层与层之间，计算期间互连处于空闲。小缓冲区加上计算期间互连空闲的事实，导致 NVLink 利用率不足。因此，消费者的流量几乎不会带来额外影响。我们相信，若 NVIDIA 公开设置数据包优先级的 API，这一开销便可消除——我们可以用它让张量并行的数据包优先于生产者-消费者流量。

#### 9.4 Aqa 的适应能力

本小节在 A100 测试床上展示 Aqa 对生产者负载变化的适应能力，以及 Aqa 在不同数据集上的可扩展性。



(a) Free memory (Llama 8B) (b) OPT-30B throughput.

图 12 Aqa 的弹性机会主义地把内存受限负载的吞吐量提升了 6×。

图中文字说明：(a) 为 Llama 8B 所在 GPU 的空闲内存（GB）随时间（秒）的变化；(b) 为 OPT-30B 的吞吐量（tokens/s），标注 2X RPS to Llama 表示把发往 Llama 的请求速率加倍。

**Aqa 的弹性。** 本实验在一台 2 块 A100 GPU 的服务器上，以 Llama 3.1 8B 为生产者、OPT-30B 为消费者运行 Aqa。我们改变发往生产者的推理请求数量。图 12a 展示服务 Llama 8B 的 GPU 上的可用空闲内存。起初，服务引擎预留全部内存以容纳到来请求的推理上下文。Aqa-lib 注意到推理负载稳定，便依据剖析数据把该 GPU 上的内存提供给消费者。在图 12a 的约 60 秒处，我们开始长提示词推理，获得每秒 8 个令牌的高吞吐量。在 120 秒处，我们把发往生产者的流量加倍，待处理队列中的请求数随之增加；Aqa-lib 注意到这一点并迅速回收内存以容纳负载。回收内存降低了消费者的吞吐量（图 12b）。当 LLM 服务完所有请求后，吞吐量再度上升。Aqa-lib 再次检测到稳定流量，并把消费者卸载的张量透明地移回生产者的 GPU。这表明 Aqa 能动态响应生产者负载的变化，机会主义地提升消费者性能。

**Aqa 的收益可延伸到其他数据集。** 本实验使用 arxiv 摘要数据集与 Yi-34B，展示 Aqa 的收益如何迁移到其他数据集。arxiv 数据集的提示词更长，中位数约 7000 令牌，而 ShareGPT 为 2000 令牌。我们先用 Aqa-profiler 为单块 A100 GPU 上的 Yi-34B 剖析应对突发所需的交换空间。给定稳定请求速率，剖析器以稳定速率的倍数制造突发，确定该模型为应对一分钟 2×、3×、5× 于稳定速率的突发分别需要 6GB、15GB、40GB 交换空间。

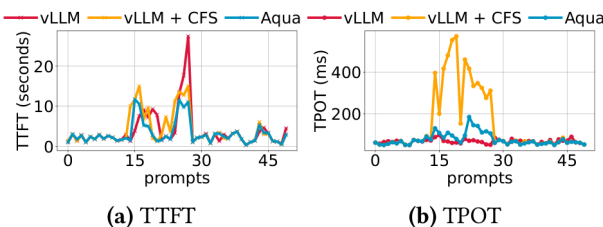


图 13a 展示按序到达单块 A100 GPU 上 Yi-34B 的请求的 TTFT，请求采样自 Arxiv 数据集。图 13b 展示这些请求的 TPOT。

图中文字说明：(a) 为 TTFT（秒），(b) 为 TPOT（毫秒）；横轴均为按序到达的提示词数量；图例 vLLM、vLLM + CFS、Aqua（即 Aqa）为三种方案。

我们先把 Llama 8B 托管在一块 GPU、Yi-34B 托管在另一块 GPU，以稳定速率发出请求。为模拟突发，我们把发往 Yi-34B 的请求速率增至三倍并持续一分钟。图 13 表明，Aqa 高效应对了突发：通过快速抢占，把峰值 TTFT 降低了 2×，把 TPOT 降低了 2×。第 10-20 个提示词的 TTFT 在所有系统中都偏高，因为提示词很长而预填充阶段主要受计算限制。

## 10 相关工作

**GPU 作业调度器。**Usher [57] 是最新的调度器，它剖析 GPU 上的计算与内存利用率，并在 GPU 上复用互补的模型以提高资源利用率。Alpa-serve [36] 通过把其他负载利用不足的 GPU 分配给高流量作业，动态调整分配给作业的 GPU 数量。类似地，Salus [68]、Clockwork [30]、Orion [18] 与 PipeSwitch [19] 在模型、层与内核粒度上复用模型，以高效分时 GPU 计算。所有这些系统在复用模型时都把 GPU 上的计算与内存分配紧耦合，错过了计算饱和时利用空闲内存容量的机会。Aqa 弥补了这一空白，即使没有共享计算的机会，也能最大化多 GPU 服务器的内存利用率。

**在线 LLM 推理的调度器。**近来，社区为到达 LLM 的提示词提出了许多调度算法 [4, 54, 59, 65]。Sarathi-serve [4] 提出了把预填充与解码合批的调度器。Llumnix [59] 通过负载均衡、自动扩容与活跃迁移，跨 LLM 实例调度提示词。VTC [54] 通过准入控制为 LLM 服务引擎提供粗粒度公平调度，防止恶意行为者压垮基础设施。VTC 还提到，抢占式调度与其工作正交，应在未来解决。Aqa 中的 CFS 与这些系统互补：在自动扩容生效期间应对突发，并为已接纳的提示词公平分配资源。MLFQ [65] 等工作同样构建抢占式调度器，而 Aqa-lib 能以更快的抢占进一步提升其性能。

**CUDA 统一虚拟内存。**CUDA 统一内存提供横跨 GPU 与主机内存的单一虚拟地址空间。CUDA 驱动管理缺页与预取，允许 GPU 在所有 GPU 与 DRAM 之间无缝分配和访问内存。然而，其默认预取策略不足以支撑 LLM 推理——推理上下文散布在不连续的内存区域。此外，CUDA 驱动不会机会主义地利用 NVLink 与 NVSwitch 互连上的空闲 GPU 内存。先前工作如 Memory Harvester [23] 在驱动层利用空闲 GPU 内存作为交换空间来应对这一局限，但它保持应用无关性，不给予负载对逐出与预取的控制权。缺乏控制可能导致低效换页，例如逐出模型权重而非推理上下文，增加开销。相比之下，LLM 服务引擎在应用层手动管理预取与逐出以优化内存使用。Aqa 改进了 CUDA 统一内存：支持显式控制预取顺序，并机会主义地把卸载的数据迁移到更快的 GPU 内存，为 LLM 推理最大化换页性能。

**其他生成式引擎。**Orca [67] 是引入连续批处理 (continuous batching) 的 LLM 推理引擎，vLLM [35] 以分页注意力 (paged attention) 的设计进一步改进。Deepspeed-zero [12] 是 FlexGen [55] 的基线。鉴于 Aqa 能提升 FlexGen 的性能，类似收益可延伸到 Deepspeed。HeteGen [69] 在 FlexGen 基础上同时利用 CPU 与 GPU 进行计算。由于 HeteGen 的 CPU 到 GPU I/O 同样依赖 PCIe，Aqa 与之互补，可进一步提升其性能。与 CacheGen [37] 用精度换压缩并把大量提示词存于远端内存不同，Aqa 在保持精度的同时缓存上下文，适合医疗、法律与金融等使用关键数据的场景。我们注意到，也有支持快速剖析的模拟器 [3]。Aqa-profiler 同样可以对接这类模拟器以更快剖析，但本文为准确性直接从硬件测量。DistServe [70] 与 Splitwise [50] 提出为预填充与解码阶段使用不同硬件。由于 Aqa-profiler 能把解耦的负载分类为生产者或消费者，Aqa 与它们互补。

**基于 RDMA 的 DRAM 页面交换。**Infiniswap [29] 与 Fastswap [10] 通过收割内存、经 RDMA 在互连服务器的 DRAM 之间移动页面，加速操作系统的换页。这些系统中的内存分配算法不保证带宽隔离，因为同一生产者可能被多个消费者共享。理解 NVLink 与 NVSwitch 这类封闭系统在争用下的行为，对提供换页带宽保证至关重要，但其拥塞控制算法闭源而难以研究。尽管 Aqa 的设计允许我们实现类似的内存分配算法 [10, 29] 来跨 GPU 分配内存，我们仍开发了 Aqa-placer，因为我们希望在换页期间提供带宽保证。我们把争用下 NVLink 换页带宽的建模留作未来工作。

## 11 致谢

本工作部分由 ACE (JUMP 2.0 的七个中心之一，由 DARPA 赞助的半导体研究公司 (SRC) 项目)、NSW 奖项 #2444537 以及 Cornell Bowers CIS-LinkedIn 基金支持。

## 参考文献

- [1] 2024. PyTorch: An open source machine learning framework that accelerates the path from research prototyping to production deployment. <https://pytorch.org>. Accessed: 2024-05-03.
- [2] Saksham Agarwal, Arvind Krishnamurthy, and Rachit Agarwal. 2023. Host Congestion Control. In *Proceedings of the ACM SIGCOMM 2023 Conference (ACM SIGCOMM '23)*. Association for Computing Machinery, New York, NY, USA, 275–287. <https://doi.org/10.1145/3603269.3604878>
- [3] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024. Vidur: A Large-Scale Simulation Framework For LLM Inference. *arXiv:2405.05465 [cs.LG]* <https://arxiv.org/abs/2405.05465>

- [4] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). USENIX Association, Santa Clara, CA, 117–134. <https://www.usenix.org/conference/osdi24/presentation/agrawal>
- [5] Mistral AI. 2023. Mistral. [https://huggingface.co/docs/transformers/main/model\\_doc/mistral](https://huggingface.co/docs/transformers/main/model_doc/mistral).
- [6] Mistral AI. 2023. Mixtral. [https://huggingface.co/docs/transformers/model\\_doc/mixtral](https://huggingface.co/docs/transformers/model_doc/mixtral).
- [7] Meta AI. 2024. CodeLLaMa. <https://github.com/Meta-Llama/codeLlama>.
- [8] Meta AI. 2024. LLaMA 3.2. <https://www.llama.com/>.
- [9] Together AI. 2024. Together: The fastest cloud platform for building and running generative AI. <https://www.together.ai/>. Accessed: 2024-03-16.
- [10] Emmanuel Amaro, Christopher Branner-Augmon, Zhihong Luo, Amy Ousterhout, Marcos K. Aguilera, Aurojit Panda, Sylvia Ratnasamy, and Scott Shenker. 2020. Can far memory improve job throughput?. In Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys '20). Association for Computing Machinery, New York, NY, USA, Article 14, 16 pages. <https://doi.org/10.1145/3342195.3387522>
- [11] Amazon Web Services. 2024. EC2 Auto Scaling Warm Pools. Accessed: 2024-10-12.
- [12] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed-inference: enabling efficient inference of transformer models at unprecedented scale. In Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC '22). IEEE Press, Article 46, 15 pages.
- [13] Anthropic. [n. d.]. Prompt Caching. <https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching>. Accessed: 2024-09-11.
- [14] arXiv. 2023. Arxiv summarization dataset. <https://huggingface.co/datasets/ccdv/arxiv-summarization>. Accessed: 2024-10-12.
- [15] Amazon Web Services (AWS). 2024. Generative AI-Use Cases and Resources. <https://aws.amazon.com/generative-ai/use-cases/>. Accessed on March 16, 2024.
- [16] Microsoft Azure. [n. d.]. Azure AI Studio. Accessed on March 16, 2024. Available at: <https://azure.microsoft.com/en-us/products/ai-studio/>.
- [17] Microsoft Azure. 2024. Azure LLM Inference Dataset 2024. <https://github.com/Azure/AzurePublicDataset/blob/master/analysis/AzureLLMInferenceDataset2024.ipynb>. Accessed: 2025-02-04.
- [18] Zhihao Bai, Zhen Zhang, Yibo Zhu, and Xin Jin. [n. d.]. Orion: Interference-aware, Fine-grained GPU Sharing for ML Applications. In EuroSys 2024.
- [19] Zhihao Bai, Zhen Zhang, Yibo Zhu, and Xin Jin. 2020. PipeSwitch: Fast Pipelined Context Switching for Deep Learning Applications. In 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). USENIX Association, 499–514. <https://www.usenix.org/conference/osdi20/presentation/bai>
- [20] Yoshua Bengio, Réjean Ducharme, and Pascal Vincent. 2000. A Neural Probabilistic Language Model. In Advances in Neural Information Processing Systems, T. Leen, T. Dietterich, and V. Tresp (Eds.), Vol. 13. MIT Press. [https://proceedings.neurips.cc/paper\\_files/paper/2000/file/72\\_Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2000/file/72_Paper.pdf)
- [21] Bob Briscoe, Anna Brunstrom, Andreas Petlund, David Hayes, David Ros, Ing-Jyh Tsang, Stein Gjessing, Gorry Fairhurst, Carsten Griwodz, and Michael Welzl. 2016. Reducing Internet Latency: A Survey of Techniques and Their Merits. Commun. Surveys Tuts. 18, 3 (July 2016), 2149–2196. <https://doi.org/10.1109/COMST.2014.2375213>
- [22] Cerebras. 2021. The future of AI is Wafer-Scale. <https://www.cerebras.net/product-chip/>.
- [23] Sangjin Choi, Taeksoo Kim, Jinwoo Jeong, Rachata Ausavarungnirun, Myeongjae Jeon, Youngjin Kwon, and Jeongseob Ahn. 2022. Memory Harvesting in Multi-GPU Systems with Hierarchical Unified Virtual Memory. In 2022 USENIX Annual Technical Conference (USENIX ATC 22). USENIX Association, Carlsbad, CA, 625–638. <https://www.usenix.org/conference/atc22/presentation/choi-sangjin>
- [24] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805 [cs.CL]
- [25] Google AI for Developers. [n. d.]. Gemini API Pricing. <https://ai.google.dev/pricing>. Accessed: 2024-09-11.
- [26] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. 2024. AI and Memory Wall. IEEE Micro 44, 3 (2024), 33–39. <https://doi.org/10.1109/MM.2024.3373763>
- [27] Google. 2024. Google user abandonment stats. Accessed: 2024-10-12.
- [28] Google. 2024. Parti-prompts. <https://huggingface.co/datasets/nateraw/parti-prompts>. Accessed: 2024-05-02.
- [29] Juncheng Gu, Youngmoon Lee, Yiwen Zhang, Mosharraf Chowdhury, and Kang G. Shin. 2017. Efficient Memory Disaggregation with Infiniswap. In 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17). USENIX Association, Boston, MA, 649–667. <https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/gu>
- [30] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 2020. Serving DNNs like Clockwork: Performance Predictability from the Bottom Up. In 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). USENIX Association, 443–462. <https://www.usenix.org/conference/osdi20/presentation/gujarati>
- [31] Gurobi. 2024. Gurobi. <https://www.gurobi.com/>. Accessed: 2024-05-03.
- [32] Intel Gaudi AI accelerator. 2021. Intel Gaudi AI accelerator. <https://habana.ai/products/gaudi/>.
- [33] Norman P. Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Cliff Young, Xiang Zhou, Zongwei Zhou, and David Patterson. 2023. TPU v4: An Optically

- Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings. arXiv:2304.01433 [cs.AR]
- [34] Felix Kreuk, Gabriel Synnaeve, Adam Polyak, Uriel Singer, Alexandre Défossez, Jade Copet, Devi Parikh, Yaniv Taigman, and Yossi Adi. 2023. AudioGen: Textually Guided Audio Generation. arXiv:2209.15352 [cs.SD]
- [35] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 611–626. <https://doi.org/10.1145/3600006.3613165>
- [36] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. [n. d.]. AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI '23)*. <https://www.usenix.org/conference/osdi23/presentation/li-zhouhan>
- [37] Yuhang Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. In *Proceedings of the ACM SIGCOMM 2024 Conference (ACM SIGCOMM '24)*. Association for Computing Machinery, New York, NY, USA, 38–56. <https://doi.org/10.1145/3651890.3672274>
- [38] Microsoft. 2024. AI Is Transforming Businesses. <https://azure.microsoft.com/en-us/blog/azure-openai-service-10-ways-generative-ai-is-transforming-businesses/>, accessed on March 16, 2024.
- [39] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient Estimation of Word Representations in Vector Space. arXiv:1301.3781 [cs.CL]
- [40] MosaicML. 2023. MPT. [https://huggingface.co/docs/transformers/main/model\\_doc/mpt](https://huggingface.co/docs/transformers/main/model_doc/mpt).
- [41] NVIDIA. 2024. NVIDIA Blackwell Architecture. <https://www.nvidia.com/en-us/data-center/technologies/blackwell-architecture/> Accessed: 2024-10-12.
- [42] NVIDIA. 2024. NVLink & NVSwitch: Fastest HPC Data Center Platform. <https://www.nvidia.com/en-us/data-center/nvlink/>. Accessed: 2024-03-16.
- [43] NVIDIA Corporation. 2024. NVIDIA B200. <https://www.nvidia.com/en-us/data-center/dgx-b200/>. Accessed: 2024-04-28.
- [44] NVIDIA Corporation. 2024. NVIDIA DGX A100 Datasheet. <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/nvidia-dgx-a100-datasheet.pdf>. Accessed: 2024-04-21.
- [45] NVIDIA Corporation. 2024. NVIDIA H100 Tensor Core GPU Datasheet. <https://resources.nvidia.com/en-us-tensor-core/nvidia-tensor-core-gpu-datasheet>. Accessed: 2024-04-21.
- [46] Nvidia DGX Systems 2021. Nvidia DGX Systems. <https://www.nvidia.com/en-us/data-center/dgx-systems/>.
- [47] OpenAI. 2024. OpenAI API. Accessed on March 16, 2024. Available at: <https://openai.com/blog/openai-api>.
- [48] OpenAI. 2024. OpenAI batch API. <https://platform.openai.com/docs/guides/batch>. Accessed: 2024-09-11.
- [49] OpenAI. 2024. OpenAI Customer Stories. <https://openai.com/customer-stories>. Accessed on March 16, 2024.
- [50] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative LLM inference using phase splitting. arXiv:2311.18677 [cs.AR] <https://arxiv.org/abs/2311.18677>
- [51] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. High-Resolution Image Synthesis with Latent Diffusion Models. arXiv:2112.10752 [cs.CV]
- [52] Amazon Web Services. [n. d.]. Amazon SageMaker. Accessed on March 16, 2024. Available at: <https://aws.amazon.com/sagemaker/>.
- [53] Amazon Web Services. 2024. Mewtant Case Study. <https://aws.amazon.com/solutions/case-studies/mewtant-case-study/>, accessed on 2024-03-16.
- [54] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. 2024. Fairness in Serving Large Language Models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, Santa Clara, CA, 965–988. <https://www.usenix.org/conference/osdi24/presentation/sheng>
- [55] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. arXiv:2303.06865 [cs.LG]
- [56] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2020. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv:1909.08053 [cs.CL] <https://arxiv.org/abs/1909.08053>
- [57] Sudipta Saha Shubha, Haiying Shen, and Anand Iyer. [n. d.]. USHER: Holistic Interference Avoidance for Resource Optimized ML Inference. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. <https://www.usenix.org/conference/osdi24/presentation/shubha>
- [58] Arjun Singh, Joon Ong, Amit Agarwal, Glen Anderson, Ashby Armistead, Roy Bannan, Seb Boving, Gaurav Desai, Bob Felderman, Paulie Germano, Anand Kanagala, Jeff Provost, Jason Simmons, Eiichi Tanda, Jim Wanderer, Urs Hölzle, Stephen Stuart, and Amin Vahdat. 2015. Jupiter Rising: A Decade of Clos Topologies and Centralized Control in Google's Datacenter Network. In *Sigcomm '15*.
- [59] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llmunix: Dynamic Scheduling for Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, Santa Clara, CA, 173–191. <https://www.usenix.org/conference/osdi24/presentation/sun-biao>
- [60] The Linux Kernel Developers. [n. d.]. CFS Scheduler. <https://docs.kernel.org/scheduler/sched-design-CFS.html>. Accessed: 2024-03-17.

- [61] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. [https://proceedings.neurips.cc/paper\\_files/paper/2017/file/3f5ee24354fdee91fbd053cfc4a845aa-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee24354fdee91fbd053cfc4a845aa-Paper.pdf)
- [62] Abhishek Vijaya Kumar, Arjun Devraj, Darius Bunandar, and Rachee Singh. 2024. A case for server-scale photonic connectivity. In *Proceedings of the 23rd ACM Workshop on Hot Topics in Networks (HotNets '24)*. Association for Computing Machinery, New York, NY, USA, 290–299. <https://doi.org/10.1145/3696348.3696856>
- [63] vLLM. 2023. Distributed LLM serving. [https://docs.vllm.ai/en/latest/serving/distributed\\_serving.html](https://docs.vllm.ai/en/latest/serving/distributed_serving.html). Accessed: 2024-10-12.
- [64] VLLM. 2024. Llama31: Enhancing LLM Serving Efficiency. <https://blog.vllm.ai/2024/07/23/llama31.html>. Accessed on September 11, 2024.
- [65] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. 2024. Fast Distributed Inference Serving for Large Language Models. *arXiv:2305.05920* [cs.LG] <https://arxiv.org/abs/2305.05920>
- [66] Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhao, Quanlu Zhang, Fan Yang, and Lidong Zhou. 2018. Gandiva: Introspective Cluster Scheduling for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, 595–610. <https://www.usenix.org/conference/osdi18/presentation/xiao>
- [67] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 521–538. <https://www.usenix.org/conference/osdi22/presentation/yu>
- [68] Peifeng Yu and Mosharaf Chowdhury. 2019. Salus: Fine-Grained GPU Sharing Primitives for Deep Learning Applications. *arXiv:1902.04610* [cs.DC]
- [69] Xuanlei Zhao, Bin Jia, Haotian Zhou, Ziming Liu, Shenggan Cheng, and Yang You. 2024. HeteGen: Heterogeneous Parallel Inference for Large Language Models on Resource-Constrained Devices. <https://arxiv.org/abs/2403.01164>. Accessed on September 11, 2024.
- [70] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 193–210. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>

## 附录 A 制品 (Artifact) 附录

### A.1 摘要

我们在 <https://github.com/aquaml> 发布 Aqa 的源代码。Aqa 需要一台至少配有 2 块经 NVLink 互连的 NVIDIA GPU 的多 GPU 服务器。Aqa 的文档见 <https://aquaml.github.io/>。

### A.2 制品清单 (元信息)

以下是制品中可用的代码仓库列表:

- 包含 Aqa 张量的 Aqa-lib;
- 带 Aqa CFS 算法的 vLLM;
- Aqa-placer;
- 集成了 Aqa-lib 以执行 stable-diffusion 的 PyTorch 脚本;
- 集成了 Aqa-lib 的 FlexGen;
- 集成了 Aqa-lib 的 Audiogen;
- 是否公开可用?: 是。